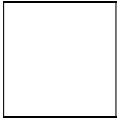
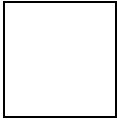


PROJECT AND TEAM INFORMATION

Project Title

Data-Structure-Based USB Malware Activity Monitor Using C and C++ OOP

Student / Team Information

<i>Team Name:</i> <i>DSCPP-III-2026-T370</i>	
Team member 1 (Team Lead)	<i>Divyanshu Panwar</i> <i>2510370085@geu.ac.in</i> <i>2028715</i> 
Team member 2	<i>Abhay Rai</i> <i>2510370531@geu.ac.in</i> <i>20289610</i> 
	
	

PROPOSAL DESCRIPTION

Motivation

USB flash drives remain a primary item for cyberattacks, especially in air-gapped systems or environments with strict network controls. Malicious actors use USB devices to deploy ransomware, spyware and exfiltrate sensitive data. Traditional antivirus software often relies on signature-based scanning, which can miss zero-day exploits or immediate malicious activity triggered upon insertion.

A dedicated, low-level monitor provides real-time visibility into the exact moment a USB device interacts with the operating system.

Current solution

There's no shortage of tools already doing pieces of this. Windows Defender, Symantec, and McAfee all scan removable media using large cloud-backed signature sets. On Linux, USBGuard handles this at the kernel level by whitelisting or blacklisting devices through udev rules. Then there's the hardware route USB firewalls and data diodes that physically block certain device classes regardless of what software says. All of these work, but they're heavy, mostly closed-source, and run as background services you can't really see inside of. That's kind of the gap we want to fill: instead of building another antivirus engine, we're building something small enough that you can actually see the detection logic working — a hash table for known devices and file signatures, a queue for events as they happen, and a tree for sorting scanned files by type. Less "black box," more "here's exactly why it flagged this."

Project Goals and Milestones

General Goals:

- Build a working USB device and file-activity monitor in C/C++ using data structures we implement ourselves.
- Show, practically, how the right data structure for the job — hash tables for lookups, queues for event handling, stacks for undo/rollback, trees for categorization — actually changes detection speed and reliability, not just in theory.
- Structure the whole thing using proper C++ OOP so it's maintainable: USBDevice, FileEvent, AlertManager, Logger as core classes.

Milestones:

1. Set up the basic classes/structs (USBDevice, FileEvent, Alert), get device insertion/removal detection working, and log events using arrays or linked lists to start.
2. Build a hash table that maps device IDs (VID:PID) and known-malicious file hashes, giving us $O(1)$ average lookup against whitelist/blacklist entries.
- 3.: Add a queue-based pipeline (FIFO) to handle device/file events as they come in, plus a stack-based quarantine feature so we can undo/rollback a suspicious file move if needed.
- 4.: Get real-time alerts working for things like mass file copying, unrecognized VID/PID combos, or an autorun.inf showing up unexpectedly. Test all of this against simulated data and fix edge cases as they show up.
5. Pull everything into one CLI (maybe a simple dashboard if time allows), write it up properly, and put together a demo.

Project Approach

The plan is to write the core data structures in C — for the low-level control and performance — and wrap them in a C++ OOP layer for structure and usability.

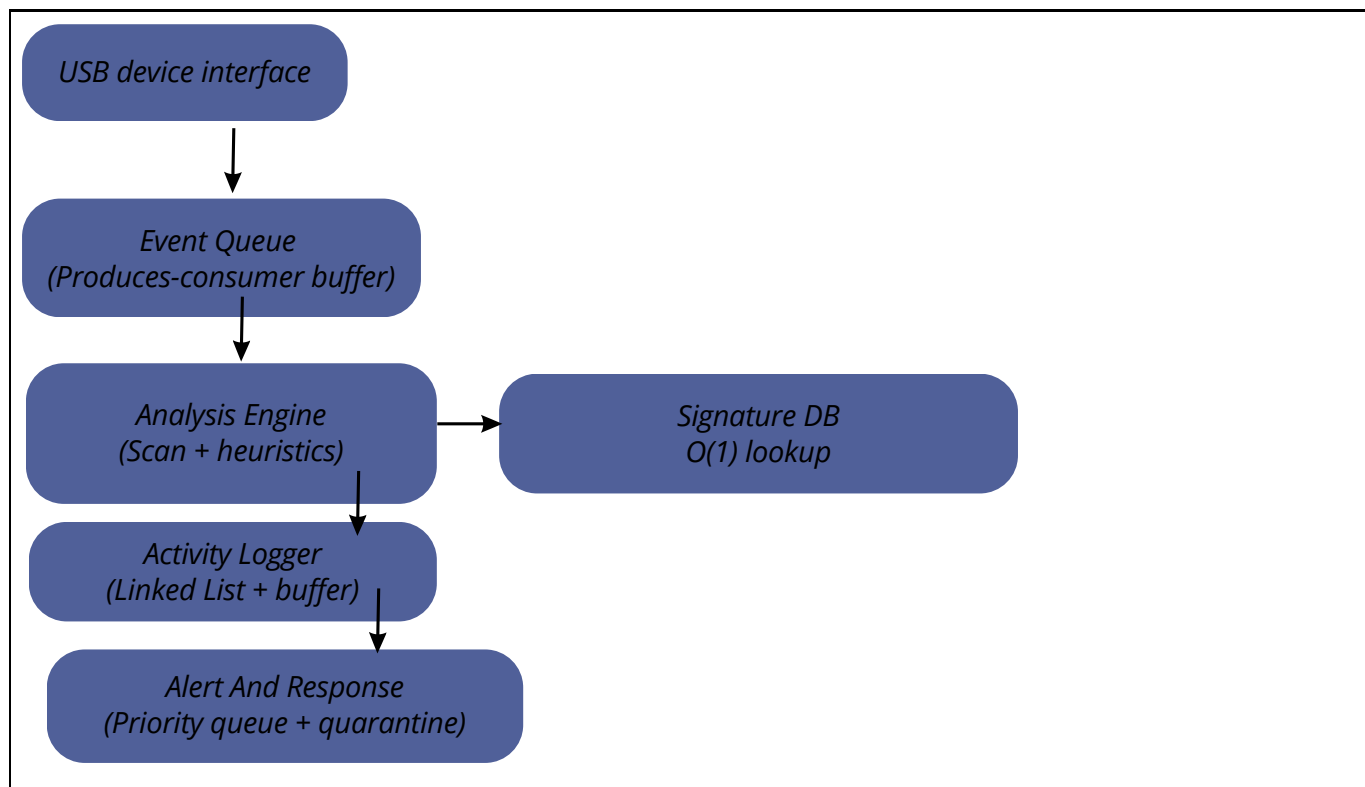
Key components:

- *Device Registry*: A hash table we build ourselves, mapping device VID:PID to device records and trust status. $O(1)$ average for lookup, insert, and delete.
- *Event Queue*: A straightforward FIFO queue processing device-attach and file-system events in the order they actually occur.
- *Quarantine Stack*: A stack that lets us undo the last N flagged operations — isolate a file, then restore it if it turns out to be a false positive.
- *File-Category Tree*: A binary search tree (or trie) organizing scanned files by extension/type, so range and prefix searches stay fast.
- *Sorting/Reporting*: Quicksort or mergesort to generate reports — "most active devices," "most frequent alert types," that kind of thing.

Platform & Technologies:

- *Language*: C (data structures) + C++ (OOP layer)
- *Dev environment*: VS Code / Code::Blocks, GCC/G++
- *Version control*: Git/GitHub
- *Optional stretch goal*: a basic dashboard (Qt, or Flask talking to a C++ backend over REST) to visualize device and alert history
- *Testing*: Simulated event logs — CSVs of device insert/remove and file-operation records — to check detection logic and compare performance across the different data structure choices. No real malware samples involved anywhere in this.

System Architecture (High Level Diagram)



Project Outcomes

By the end, we should have a working console app — call it SentinelUSB — that catches USB insertion/removal in real time, flags suspicious file operations against a hash-based whitelist/blacklist, logs everything as it happens, lets you quarantine and roll back flagged files, and spits out activity reports (most active devices, alert-type breakdowns). It'll lean on hash tables, queues, stacks, and BSTs throughout, all organized behind a clean C++ class hierarchy (USBDevice, FileEvent, AlertManager, Logger).

What we're handing in: the full source code on GitHub, a sample simulated event-log CSV we used for testing, a report explaining our design decisions and why we chose the data structures we did, and a live demo covering device monitoring, event processing, and reporting.

Assumptions

We have assumed that single-machine, single-user setup — needs no fleet monitoring across a network. Device and file events will come from simulated logs/CSVs rather than actual low-level OS hooks, which keeps things portable and safe to demo without needing special privileges. Dataset sizes should stay moderate (up to a few thousand events), so keeping everything in memory is fine — no need for disk-backed structures. And to be clear: this is strictly about detection, logging, and alerting. We're not creating, running, or distributing any actual malicious code, and no real malware samples are touched at any point.

References

- *GeeksforGeeks — data structure implementation guides*
- *Deitel & Deitel, C++ How to Program*
- *USBGuard project documentation — policy-based USB device authorization*
- *NIST SP 800-83 — Guide to Malware Incident Prevention and Handling *